

1.º Examen parcial (R) – 25/06/2026

- Recordamos que la secuencia de Fibonacci está definida como $F(0) = F(1) = 1$ y $F(n) = F(n-1) + F(n-2)$ en otro caso. Ahora bien, existe la siguiente equivalencia (por fuera de los casos base), con $k = \lfloor \frac{n}{2} \rfloor$ (parte entera de la mitad de n):
 - $F(n) = F(k) \cdot (2F(k+1) - F(k))$ si n es par,
 - $F(n) = F(k+1)^2 + F(k)^2$ si n es impar.

Usando esto, implementar un algoritmo que, utilizando **división y conquista** permita obtener el n -ésimo número de la secuencia de Fibonacci en $\mathcal{O}(\log n)$. Justificar adecuadamente la complejidad del algoritmo implementado.

Ayudas:

- Considerará hacer que la función recursiva devuelva tanto $F(n)$ como $F(n+1)$.
 - Recordá que si n es par, entonces el valor de k es igual tanto para n como para $n+1$, y que si n es impar, entonces el valor de k es igual tanto para n como para $n-1$, y que la definición de la secuencia de Fibonacci sigue valiendo ($F(i) = F(i-1) + F(i-2)$).
- Planteamos otra variante del problema de *interval scheduling*. Tenemos una secuencia de charlas a dar con horario de inicio y fin (**fin** > **inicio**). En este caso, queremos dar **todas** las charlas, por lo que necesitaremos más de un auditorio/aula para darlas, y que no hayan solapamientos. Nos interesa minimizar cantidad de aulas que necesitamos para dar **todas** las charlas, de tal forma que en una misma aula no hayan dos charlas que se solapen entre sí. Implementar un algoritmo que devuelva la conformación de estas aulas.

Este problema puede resolverse de forma óptima, y se espera que así se resuelva. ¿El algoritmo da siempre la solución óptima? Si lo hace, justificar. Si no lo es, dar un contraejemplo. Justificar por qué es un algoritmo Greedy. Indicar y justificar la complejidad.
 - Implementar un algoritmo que, dado un grafo no dirigido, obtenga el clique de mayor tamaño dentro del mismo (utilizando **backtracking**).
 - Patricio es el hermano de Juan (sí, el vago), y tiene una filosofía muy parecida a la de él. El problema es que la lleva a un extremo: no le interesa en lo más mínimo el dinero. Si fuera por Patricio, no agarraría la pala ningún día de su vida pero, por mala suerte, la vida no es tan sencilla. Su jefe ya le ha llamado la atención, por lo que sabe que no puede estar dos días seguidos sin trabajar. Tiene un arreglo de n valores, donde el elemento i indica el *esfuerzo* de trabajar el día i (cada día demanda un esfuerzo diferente). Patricio puede elegir trabajar un día, o no trabajar.

Implementar un algoritmo que, por **programación dinámica**, obtenga el mínimo esfuerzo total (suma de esfuerzos) a trabajar en el período de n días, con la restricción que Patricio no puede estar dos días seguidos sin trabajar. También escribir el algoritmo que permita reconstruir la solución. Indicar y justificar la complejidad del algoritmo implementado (el de programación dinámica, y también el de la reconstrucción).

1.º Examen parcial (R) – 25/06/2026

- Recordamos que la secuencia de Fibonacci está definida como $F(0) = F(1) = 1$ y $F(n) = F(n-1) + F(n-2)$ en otro caso. Ahora bien, existe la siguiente equivalencia (por fuera de los casos base), con $k = \lfloor \frac{n}{2} \rfloor$ (parte entera de la mitad de n):
 - $F(n) = F(k) \cdot (2F(k+1) - F(k))$ si n es par,
 - $F(n) = F(k+1)^2 + F(k)^2$ si n es impar.

Usando esto, implementar un algoritmo que, utilizando **división y conquista** permita obtener el n -ésimo número de la secuencia de Fibonacci en $\mathcal{O}(\log n)$. Justificar adecuadamente la complejidad del algoritmo implementado.

Ayudas:

- Considerará hacer que la función recursiva devuelva tanto $F(n)$ como $F(n+1)$.
 - Recordá que si n es par, entonces el valor de k es igual tanto para n como para $n+1$, y que si n es impar, entonces el valor de k es igual tanto para n como para $n-1$, y que la definición de la secuencia de Fibonacci sigue valiendo ($F(i) = F(i-1) + F(i-2)$).
- Planteamos otra variante del problema de *interval scheduling*. Tenemos una secuencia de charlas a dar con horario de inicio y fin (**fin** > **inicio**). En este caso, queremos dar **todas** las charlas, por lo que necesitaremos más de un auditorio/aula para darlas, y que no hayan solapamientos. Nos interesa minimizar cantidad de aulas que necesitamos para dar **todas** las charlas, de tal forma que en una misma aula no hayan dos charlas que se solapen entre sí. Implementar un algoritmo que devuelva la conformación de estas aulas.

Este problema puede resolverse de forma óptima, y se espera que así se resuelva. ¿El algoritmo da siempre la solución óptima? Si lo hace, justificar. Si no lo es, dar un contraejemplo. Justificar por qué es un algoritmo Greedy. Indicar y justificar la complejidad.
 - Implementar un algoritmo que, dado un grafo no dirigido, obtenga el clique de mayor tamaño dentro del mismo (utilizando **backtracking**).
 - Patricio es el hermano de Juan (sí, el vago), y tiene una filosofía muy parecida a la de él. El problema es que la lleva a un extremo: no le interesa en lo más mínimo el dinero. Si fuera por Patricio, no agarraría la pala ningún día de su vida pero, por mala suerte, la vida no es tan sencilla. Su jefe ya le ha llamado la atención, por lo que sabe que no puede estar dos días seguidos sin trabajar. Tiene un arreglo de n valores, donde el elemento i indica el *esfuerzo* de trabajar el día i (cada día demanda un esfuerzo diferente). Patricio puede elegir trabajar un día, o no trabajar.

Implementar un algoritmo que, por **programación dinámica**, obtenga el mínimo esfuerzo total (suma de esfuerzos) a trabajar en el período de n días, con la restricción que Patricio no puede estar dos días seguidos sin trabajar. También escribir el algoritmo que permita reconstruir la solución. Indicar y justificar la complejidad del algoritmo implementado (el de programación dinámica, y también el de la reconstrucción).